

SIMATS ENGINEERING

CO5-ASSESSMENT TOOL1-Design Task

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192511061
Name	Vidhushini.T.S

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 40%

Time Duration: 1 Hour

QUESTIONS: 5

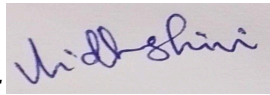
Total Marks:50

Code of Conduct:

I Vidhushini.T.S (192511061) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



1. End-to-End System Design

Design a scalable AI-based system using **PySpark and RDD operations** to analyze real-time social media data and generate intelligent recommendations, including system architecture, data flow, and agent integration.

2. Distributed Intelligent Agent System

Develop a **multi-agent AI system architecture** that uses PySpark for distributed data processing and enables agents to collaborate in solving a large-scale optimization problem.

3. Big Data AI Pipeline Design

Design a complete **distributed AI pipeline** using RDD transformations for data preprocessing, feature engineering, and model training, ensuring scalability and fault tolerance.

4. Real-Time Decision-Making System

Construct an AI-based system integrating **RDD-based streaming data processing** with intelligent agents for real-time decision-making in applications such as smart traffic or healthcare.

5. Cloud-Based Distributed AI Application

Design a **cloud-enabled AI system** using PySpark and intelligent agent architectures that supports large-scale data analytics, dynamic resource allocation, and adaptive learning.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined

Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

QUESTION 1 – End-to-End System Design

Design a scalable AI-based system using PySpark and RDD operations to analyze real-time social media data and generate intelligent recommendations, including system architecture, data flow, and agent integration.

Solution:

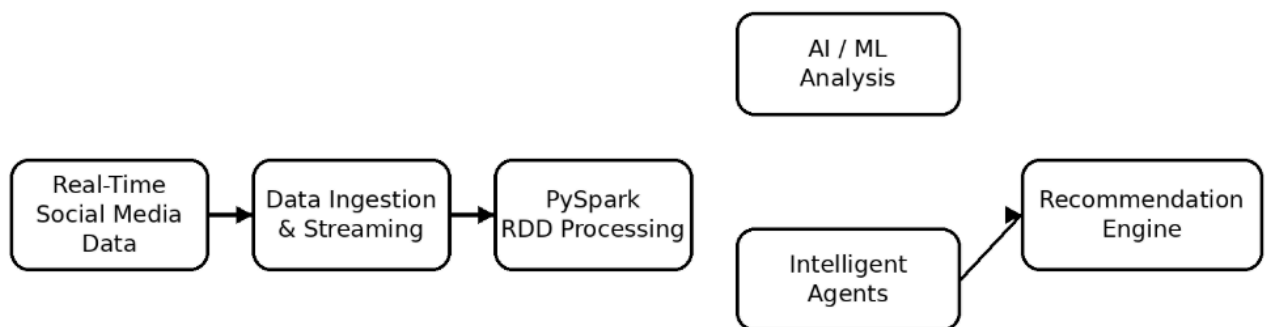
Social media platforms generate huge amounts of data in the form of posts, comments, likes, shares, hashtags, and user interactions. Processing this continuously generated data using traditional systems can be time-consuming. Therefore, a scalable distributed AI system is required.

The proposed system uses PySpark and RDD operations to process large-scale social media data. Intelligent agents analyze user behavior, identify trends, perform sentiment analysis, and generate personalized recommendations.

Objectives

1. To collect real-time social media data.
2. To process large volumes of data using PySpark.
3. To use RDD transformations and actions.
4. To identify trending topics and keywords.
5. To analyze user interests and behavior.
6. To perform sentiment analysis.
7. To integrate multiple intelligent agents.
8. To generate personalized recommendations.
9. To provide scalable and fault-tolerant processing.
10. To support real-time decision-making.

3. System Architecture



4. Data Collection

Data	Example
User ID	U1025
Post	AI is changing the world
Hashtag	#ArtificialIntelligence
Likes	250
Shares	80
Comments	45
Timestamp	10:30 AM
Sentiment	Positive

The incoming data is continuously sent to the data ingestion layer.

5. PySpark Processing

PySpark provides distributed processing capabilities. The social media records are converted into RDDs.

```
from pyspark import SparkContext
```

```
sc = SparkContext("local", "SocialMediaAnalysis")
```

```
data = [  
    "AI is changing technology",  
    "Machine learning is useful",  
    "I love artificial intelligence"  
]
```

```
rdd = sc.parallelize(data)
```

The data can then be distributed across multiple worker nodes.

6. RDD Transformations

The important transformations are:

`map()` – modifies each record.

```
lower = rdd.map(lambda x: x.lower())
```

filter() – removes unwanted records.

```
filtered = rdd.filter(lambda x: len(x) > 10)
```

flatMap() – divides posts into individual words.

```
words = rdd.flatMap(lambda x: x.split())
```

distinct() – obtains unique values.

```
unique_words = words.distinct()
```

reduceByKey() – calculates frequency.

```
count = words.map(lambda x: (x,1)) \  
    .reduceByKey(lambda a,b: a+b)
```

7. RDD Actions

Actions produce final results.

Examples:

```
count.collect()
```

```
rdd.count()
```

```
rdd.take(10)
```

Thus, transformations process the data while actions return the results.

8. Intelligent Agents

The system contains several agents.

Data Analysis Agent

- Processes social media information.
- Extracts important keywords.
- Calculates frequencies.

Trend Detection Agent

- Identifies trending topics.
- Detects popular hashtags.
- Finds rapidly increasing topics.

User Preference Agent

- Studies user interactions.
- Identifies user interests.
- Creates a user preference profile.

Sentiment Agent

- Determines positive, negative, or neutral sentiment.
- Studies user reactions.

Recommendation Agent

- Combines information from all agents.
- Generates personalized recommendations.

9. Data Flow

Social Media Data



Data Ingestion



PySpark



RDD Creation



Cleaning & Filtering



Feature Extraction



AI Analysis



Intelligent Agents



Recommendation Engine



Personalized Recommendation

Scalability and Fault Tolerance: The system can process increasing data volumes by adding more worker nodes. RDDs are divided into partitions and processed in parallel. RDD lineage also helps Spark recover lost partitions by recomputing them from previous operations.

Advantages:

- Handles large datasets.
- Supports distributed processing.
- Provides personalized recommendations.
- Supports real-time analysis.
- Uses parallel processing.
- Provides fault tolerance.
- Detects trends automatically.
- Can be scaled easily.

QUESTION 2 – Distributed Intelligent Agent System

Develop a multi-agent AI system architecture that uses PySpark for distributed data processing and enables agents to collaborate in solving a large-scale optimization problem.

Solution :

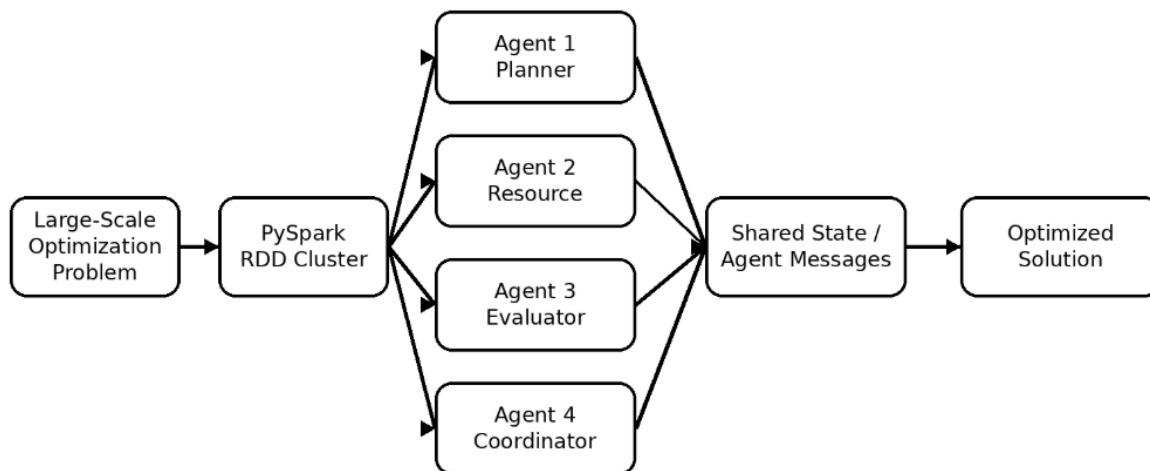
Large-scale optimization problems involve a huge number of variables and constraints. Examples include timetable scheduling, traffic routing, resource allocation, supply-chain optimization, and task scheduling.

A centralized system may require significant processing time. A multi-agent distributed system divides the problem among several intelligent agents. PySpark is used to process the data in parallel.

Objectives

1. To design a distributed multi-agent system.
2. To use PySpark for large-scale processing.
3. To divide optimization tasks among agents.
4. To enable communication between agents.
5. To improve processing speed.
6. To support parallel execution.
7. To provide fault tolerance.
8. To generate an optimized solution.

3. System Architecture



Problem Input

- Subjects
- Faculty
- Classrooms
- Student groups
- Time slots
- Faculty availability
- Room capacity

PySpark RDD Processing

The input data is converted into RDDs.

```
data_rdd = sc.parallelize(optimization_data)
```

The RDD can be partitioned across multiple worker nodes.

```
partitioned_data = data_rdd.repartition(4)
```

Different partitions can be processed simultaneously.

Intelligent Agents:

Planning Agent: Creates possible solutions and schedules.

Resource Agent: Checks available resources such as rooms, machines, or employees.

Evaluation Agent: Evaluates solutions based on constraints and objective functions.

Coordinator Agent: Coordinates the agents and selects the best solution.

Optimization Process:

The system follows these steps:

1. Receive the optimization problem.
2. Convert the input into RDDs.
3. Divide the problem into smaller tasks.
4. Assign tasks to agents.
5. Agents independently analyze possible solutions.
6. Results are returned to the coordinator.
7. Conflicts are identified.
8. Solutions are evaluated.
9. The best solution is selected.
10. Final result is generated.

Example:

Consider classroom allocation.

The Planning Agent creates possible schedules.

The Resource Agent checks whether classrooms are available.

The Evaluation Agent checks:

- Room capacity.
- Faculty availability.
- Time conflicts.
- Student conflicts.

The Coordinator Agent compares all solutions and selects the schedule with minimum conflicts.

QUESTION 3 – Big Data AI Pipeline Design

Design a complete distributed AI pipeline using RDD transformations for data preprocessing, feature engineering, and model training, ensuring scalability and fault tolerance.

Solution:

Big Data applications require systems capable of processing massive datasets efficiently. A traditional single-machine system may not provide sufficient processing speed or memory.

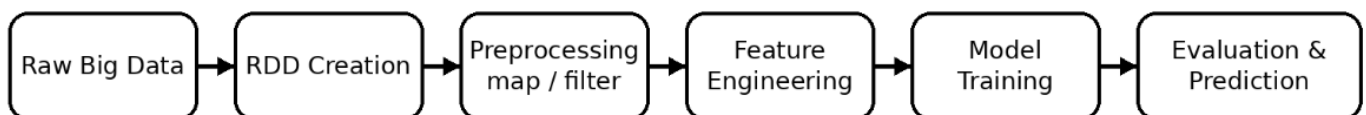
A distributed AI pipeline using PySpark RDDs can divide the data across multiple machines and process it in parallel.

2. Objectives

The proposed pipeline aims to:

- Collect large datasets.
- Clean and preprocess the data.
- Remove unwanted records.
- Transform raw data.
- Extract useful features.
- Train an AI model.
- Evaluate the model.
- Provide predictions.
- Maintain scalability.
- Provide fault tolerance.

3. Architecture



4. Data Collection

The input can come from:

- Databases.
- Sensors.
- Web applications.
- Social media.
- Enterprise systems.
- Log files.

The collected data is converted into RDDs.

```
rdd = sc.textFile("data.csv")
```

5. Data Preprocessing

Raw data may contain:

- Missing values.
- Duplicate records.
- Incorrect values.
- Unnecessary fields.
- Noise.

RDD transformations can be used for preprocessing.

```
clean_rdd = rdd.filter(lambda x: x.strip() != "")
```

Duplicate records can be removed using:

```
unique_rdd = clean_rdd.distinct()
```

6. Feature Engineering

Feature engineering converts raw information into useful numerical or categorical features.

For example:

Raw Data

↓

Age

Income

Purchase History

↓

Feature Vector

↓

Machine Learning Model

Important features are selected based on their relevance to the prediction task.

7. RDD Transformations

Important transformations include:

`map()`

Used for converting records.

`filter()`

Used for selecting relevant records.

`flatMap()`

Used for extracting multiple values.

`distinct()`

Used for removing duplicates.

`reduceByKey()`

Used for aggregation.

`groupByKey()`

Used for grouping records.

8. Model Training

After feature engineering, the prepared data is supplied to a machine learning algorithm.

The distributed environment allows large datasets to be processed efficiently.

The training process can be:

Training Data



Feature Extraction



Distributed Processing



ML Model Training



Trained Model

9. Model Evaluation

The trained model is evaluated using measures such as:

- Accuracy.
- Precision.
- Recall.
- F1-score.
- Mean Squared Error.

The evaluation results determine whether the model is suitable for prediction.

10. Scalability

RDDs are divided into partitions.

11. Fault Tolerance

RDDs are fault tolerant because Spark stores lineage information about transformations.

If a partition is lost, Spark can recompute that partition instead of restarting the entire process.

QUESTION 4 – Real-Time Decision-Making System

Construct an AI-based system integrating RDD-based streaming data processing with intelligent agents for real-time decision-making in applications such as smart traffic or healthcare.

1. Introduction

Real-time applications continuously generate data and require immediate decisions. Smart traffic systems and healthcare monitoring systems are examples where delayed decisions can reduce system effectiveness.

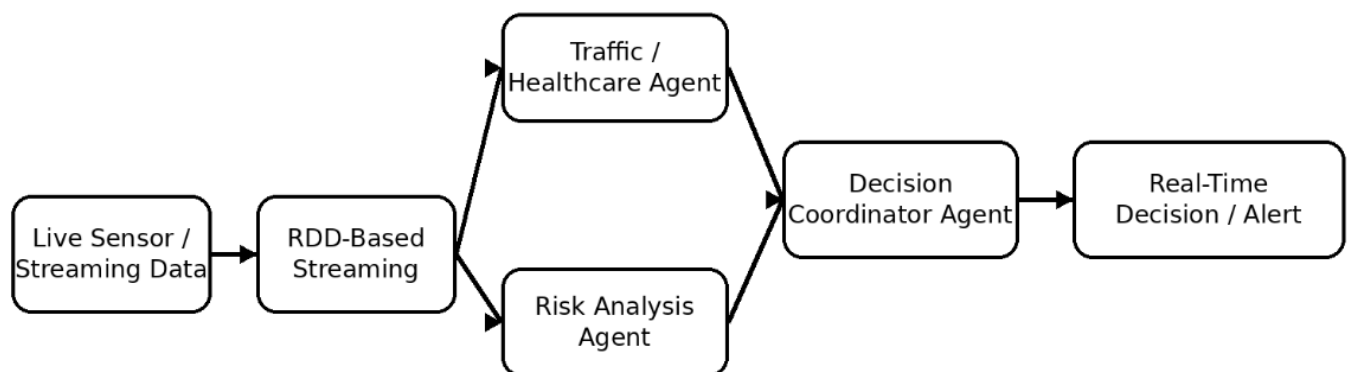
The proposed system combines streaming data processing, PySpark, RDD-based processing, and intelligent agents.

2. Objectives

The objectives are:

1. To collect real-time data.
2. To process streaming data.
3. To use RDD-based processing.
4. To identify important events.
5. To use intelligent agents.
6. To make real-time decisions.
7. To generate alerts.
8. To reduce response time.

3. Architecture



4. Example – Smart Traffic System

Traffic information can be collected using:

- Traffic cameras.
- Road sensors.
- GPS devices.
- Vehicle counters.
- Traffic signals.

Example data:

Parameter	Example
Road ID	R102
Vehicle Count	150
Average Speed	18 km/h
Traffic Level	High
Timestamp	10:30 AM

5. Streaming Data Processing

The continuously arriving data is processed using the distributed processing layer.

The data is converted into RDD-based records and processed across worker nodes.

Example:

```
traffic_rdd = sc.parallelize(traffic_data)
```

```
high_traffic = traffic_rdd.filter(  
    lambda x: x["vehicle_count"] > 100  
)
```

6. Intelligent Agents

Traffic Analysis Agent

Analyzes traffic conditions.

Risk Analysis Agent

Identifies accident risks and congestion.

Decision Coordinator Agent

Combines results and determines the appropriate action.

7. Decision Process

Traffic Data



RDD Processing



Traffic Analysis



Risk Detection



Agent Collaboration



Decision Coordinator



Change Signal Timing / Alert

8. Example Decision

Suppose Road A has:

- 200 vehicles.
- Low average speed.
- Increasing congestion.

The Traffic Analysis Agent detects heavy traffic.

The Risk Analysis Agent determines that congestion is increasing.

The Decision Coordinator may recommend:

"Increase green-light duration for Road A."

This decision can then be sent to the traffic control system.

9. Real-Time Healthcare Extension

The same architecture can be applied to healthcare.

Sensors can monitor:

- Heart rate.
- Temperature.
- Oxygen level.
- Blood pressure.

If abnormal values are detected, the intelligent agent can generate an alert.

10. Scalability

Multiple worker nodes can process different streams simultaneously.

This allows the system to handle thousands or millions of sensor records.

11. Fault Tolerance

If one worker fails, Spark can recover lost RDD partitions through lineage-based recomputation.

QUESTION 5 – Cloud-Based Distributed AI Application

Design a cloud-enabled AI system using PySpark and intelligent agent architectures that supports large-scale data analytics, dynamic resource allocation, and adaptive learning.

1. Introduction

Cloud computing provides scalable computing resources that can be dynamically increased or decreased according to application requirements. When combined with PySpark and intelligent agents, it can provide an efficient environment for large-scale AI applications.

The proposed system uses cloud storage, distributed PySpark processing, intelligent agents, and adaptive learning.

2. Objectives

The objectives are:

- To design a cloud-based AI system.
- To process large datasets using PySpark.
- To provide distributed computing.
- To dynamically allocate resources.
- To support adaptive learning.
- To provide scalable data analytics.
- To reduce processing time.
- To improve resource utilization.

3. Cloud Data Sources

Data can be collected from:

- Websites.
- IoT devices.
- Enterprise databases.
- Social media.
- Applications.
- Cloud services.

The data is stored in cloud storage before processing.

4. Resource Allocation Agent

The Resource Allocation Agent continuously monitors:

- CPU usage.
- Memory usage.
- Number of tasks.
- Worker availability.
- Processing time.

If the workload increases, additional resources can be allocated.

If the workload decreases, unnecessary resources can be released.

5. Analytics / Learning Agent

This agent analyzes processed data and performs AI/ML operations.

Responsibilities include:

- Feature analysis.
- Pattern detection.
- Prediction.
- Model evaluation.
- Learning from new data.

6. Adaptation Agent

The Adaptation Agent monitors system performance and modifies the system according to changing conditions.

For example:

Workload increases

↓

Adaptation Agent detects change

↓

Resource Agent increases resources

↓

PySpark adds processing capacity

↓

Processing continues efficiently

7. Dynamic Resource Allocation

Dynamic resource allocation is an important advantage of cloud computing.

If the workload is low:

Low Workload → Fewer Workers

If the workload increases:

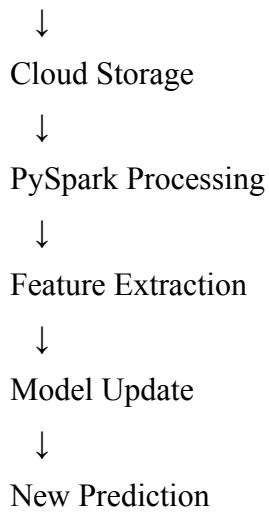
High Workload → More Workers

This helps reduce unnecessary resource usage.

8. Adaptive Learning

The AI model can continuously learn from newly available data.

New Data



This allows the system to adapt to changing patterns.

9. Scalability

Cloud resources can be increased when the data volume grows.

The system supports:

- Horizontal scaling.
- Parallel processing.
- Distributed storage.
- Dynamic resource allocation.
- Load distribution.

10. Fault Tolerance

If a worker node fails, distributed processing can continue using other nodes. Spark can also recompute lost RDD partitions using lineage information.

11. Advantages

1. Highly scalable.
2. Distributed processing.
3. Dynamic resource allocation.
4. Supports large datasets.
5. Adaptive learning.
6. Fault tolerant.
7. Efficient resource utilization.
8. Suitable for cloud environments.
9. Supports parallel processing.
10. Reduces processing time.